

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Aim

To study and implement Java Servlet concepts including form processing, session management using HttpSession and Cookies, and JDBC-based database connectivity for dynamic web applications.

Question 1 – Student Registration using Servlet

Servlet Architecture

A servlet-based application consists of a client browser, web server, web container and Java Servlet. The client sends an HTTP request to the server. The web container identifies the servlet, creates request and response objects, and executes the servlet. The response is then returned to the client.



Servlet Life Cycle

init() – Called once when the servlet is initialized.

service() – Processes client requests and calls the appropriate doGet() or doPost() method.

destroy() – Called before the servlet is removed and is used to release resources.

GET vs POST

GET	POST
Data appears in URL.	Data is sent in request body.
Suitable for retrieving/searching data.	Suitable for submitting sensitive or large form data.
Not suitable for passwords.	Preferred for passwords and registration data.

Working Student Registration Demo

Name

varsha

Email

varsha.s.puttu@gmail.com

Submit using GET

Submit using POST

GET Request: Student varsha with email varsha.s.puttu@gmail.com submitted successfully.

© 2020

Servlet Code

```
@WebServlet("/student")

public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");

        response.getWriter().println(
            "Student: " + name + " Email: " + email
        );
    }

    protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");

        response.getWriter().println(
            "Registration successful for " + name
        );
    }
}
```

Decision: POST is more appropriate for sensitive registration information because data is sent in the request body. However, HTTPS is still required for secure transmission.

Question 2 – Session Management

Session Tracking

HTTP is stateless, so session tracking is required to identify a user across multiple requests. HttpSession stores user-specific information on the server, while cookies store small pieces of information in the client browser.

Working Login Demo

Username

Password

Login successful! Session and cookie created for varsha

Cookie Example

```
Cookie cookie =  
    new Cookie("username", "Student");  
  
response.addCookie(cookie);
```

Session vs Cookies

Feature	HttpSession	Cookies
Storage	Server	Client browser
Security	Better for sensitive session data	Should not contain sensitive information
Usage	Login, shopping cart	Preferences, identifiers
Resource	Uses server memory	Uses client storage

Question 3 – JDBC Product Database

JDBC Architecture

JDBC (Java Database Connectivity) is an API that allows Java applications to communicate with relational databases. It uses JDBC drivers to connect the Java application with the database.



JDBC Connectivity Steps

1. **Load Driver:** Load the appropriate JDBC driver.
2. **Establish Connection:** Use DriverManager to connect to the database.
3. **Create Statement:** Create PreparedStatement or Statement.
4. **Execute SQL:** Execute INSERT, SELECT, UPDATE or DELETE queries.
5. **Process Result:** Read data using ResultSet.
6. **Close Resources:** Close ResultSet, Statement and Connection.

Product Database Example

Product Name

Price

Insert Product

Product 'bottle' with price ₹200 is ready to be inserted into the JDBC database.

SQL Queries

```
CREATE TABLE products (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE  
);  
  
INSERT INTO products(name, price)  
VALUES ('Laptop', 55000);  
  
SELECT * FROM products;
```